Öğrenme fazları: 1 • Ön-öğretim | 2 • İçerik | 3 • Katılım | 4 • Değerlendirme | 5 • İzleme

1 • Ön-öğretim **Ön Bilgi:** Hf.3'ün `Promise/async/await` bilgisi (`fetch` de bir Promise kaynağıdır) ve Hf.5'in `useEffect` + bağımlılık dizisi + temizleme fonksiyonu bilgisi bu haftanın `useVeri` hook'unda doğrudan kullanılır; Hf.6'nın tüm rotaları, durum kodları ve camelCase JSON sözleşmesi değişmeden, olduğu gibi tüketilir.

» Ünite 1 — `fetch` ve Tiplenmiş İstemci

`fetch` ve `Response`

`await fetch(url)` bir HTTP isteği başlatır, Hf.3'ün Promise/`async/await` sözdizimi aynen kullanılır. `response.ok` durum kodu 200–299 aralığındaysa `true`; `response.status` sayısal durum kodu; `response.json()` gövdeyi ayrıştıran ayrı bir Promise döndürür — iki ayrı `await` adımı gerekir.

Tiplenmiş İstemci

`apiGet<T>(yol)` ve `apiPost<TIn,TOut>(yol,govde)` — fetch + `response.ok` kontrolü + `json()` adımlarını tek bir yerde toplar; Hf.3'ün `Etkinlik/Koltuk/Bilet` tipleri camelCase sözleşmesi sayesinde hiçbir dönüşüm yazılmadan uygulanır.

`ApiHatasi` Sınıfı

`response.ok` `false` olduğunda `apiGet`/`apiPost` bu sınıftan bir örneği `throw` eder; çağıran kod `e instanceof ApiHatasi` ile `e.status` durum koduna göre ayrım yapabilir.

Ortak Hata / Tuzak

Ağ hatası ile 4xx/5xx cevabını karıştırmak: `fetch` yalnızca sunucuya hiç ulaşılamadığında reddedilir (reject); bir `404`/`409` gelmesi Promise'i normal çözer — `try/catch` bunu yakalamaz, daima `response.ok` kontrol edilmelidir.

Tiplenmiş `apiGet` sarmalayıcısı (`api_get` etiketi):

```
export async function apiGet<T>(
  yol: string,
  { signal }: IstekSecenekleri = {},
): Promise<T> {
  const response = await fetch(`${TABAN_URL}${yol}`, { signal });
  return cevabiIsle<T>(response);
}
```

» Ünite 2 — Yükleme/Hata/Boş Durumları

Üç Durumlu Model (*discriminated union*)

Ortak bir `tur` alanıyla ayırt edilen `'yukleniyor'`/`'hata'`/`'basarili'` durumlarını taşıyan `VeriDurumu<T>` tipi — Hf.3'ün tip daraltma bilgisinin genişletilmiş biçimi; `durum.tur === 'basarili'` kontrolünden sonra `durum.veri`'ye erişim güvenlidir.

`useEffect` İçinde Veri Çekme

`useEffect`'in kendisi `async` olamaz (Promise döndüremez); efekt içinde ayrı bir `async` fonksiyon tanımlanıp hemen çağrılır — Hf.5'in "veri çekme de bir yan etkidir" cümlesinin gerçekleştiği yer.

Yarış Durumu ve `AbortController`

İki hızlı ardışık istek (ör. `/etkinlikler/3` sonra `/etkinlikler/7`) yanlış sırayla cevap verebilir; efekt içinde bir `AbortController` oluşturup temizleme fonksiyonunda `controller.abort()` çağırmak önceki isteği iptal eder, eski cevap state'i artık güncellemez.

`useVeri<T>` Özel Hook

Hf.5'in özel hook kalıbının (`useSepet`, `useGeriSayim`) yeni bir örneği — `useEffect`+`useState`+`AbortController` mantığını tek yerde toplar; her sayfa yalnızca `const durum = useVeri<Etkinlik[]>(...)` yazar — iyimser güncelleme (sunucu cevabını beklemeden arayüzü önce güncelleme, `409`'da geri alma) Ünite 4'te bu hook'un üzerine kurulur.

» Ünite 3 — CORS İlk Temas

Origin (Köken)

protokol + alan adı + port üçlüsü — `http://localhost:5173` ile `http://127.0.0.1:8000` yalnızca port farkıyla bile farklı iki origin'dir.

Aynı-Origin Politikası

Tarayıcının varsayılan davranışı: bir sayfanın kendi origin'i dışındaki bir adrese yaptığı `fetch` isteğinin cevabını JavaScript koduna okutturmaması — bir güvenlik önlemidir (ayrıntı Hf.11).

Preflight `OPTIONS` İsteği

JSON gövdeli `POST`/`PUT`/`PATCH`/`DELETE` gibi "basit olmayan" istekler için tarayıcı asıl istekten önce otomatik bir `OPTIONS` isteği gönderir — "bu origin'den, bu metotla istek atabilir miyim?" diye sorar.

`CORSMiddleware` ve İzinli Köken Başlığı

`app.add_middleware(CORSMiddleware, allow_origins=[...])` sunucunun cevabına hangi origin'lerin cevabı okuyabileceğini belirten `Access-Control-Allow-Origin` başlığını ekler; `allow_origins` yalnızca geliştirme adresini içermeli, `["*"]` kötü bir alışkanlıktır (Hf.11) — Vite'ın `server.proxy` seçeneği CORS'u baypas edebilir ama CORS'un kendisini anlamak amaçlandığı için bu kitapta kullanılmaz.

Origin başlığıyla test edilen CORS davranışı (`test_cors_izinli_origin` etiketi):

```
def test_cors_izinli_origin(client):
    cevap = client.get(
        "/etkinlikler", headers={"Origin": "http://localhost:5173"}
    )
    assert cevap.status_code == 200
    assert cevap.headers["access-control-allow-origin"] == "http://
        localhost:5173"
```

» Ünite 4 — React Router: Sayfalar, Parametreli Rotalar

`BrowserRouter`/`Routes`/`Route`

Uygulamanın tamamını saran `BrowserRouter` kök bileşeni tarayıcının `history` API'sini kullanır; `Routes` içindeki `Route`'lar sırayla değerlendirilir, ilk eşleşen yol render edilir.

Gezinme: `Link` vs `useNavigate`

`<Link to="/etkinlikler/3">` tıklandığında tarayıcıya yeni bir HTTP isteği göndertmez, adres çubuğunu güncelleyip eşleşen bileşeni render eder — SPA gezinmesinin somut kanıtı; `navigate('/biletlerim')` ise bir olay işleyicisi içinden (ör. form başarıyla gönderildikten sonra) programatik gezinme yapar — `Link`'in aksine bir tıklama gerektirmez.

Parametreli Rota ve `useParams`

`:id` URL'nin değişken bir parçasını yakalar (`path="etkinlikler/:id"`); `useParams` ile erişilir — `id` her zaman `string`'dir, `apiGet`'e geçmeden önce `Number(id)` ile çevrilir.

`Outlet`, Sayfa Düzeni ve 404

`Duzen.tsx` bir nav çubuğu + `<Outlet />` içeren sarmal bileşendir; `<Route element={<Duzen />}>` içine yerleştirilen iç `Route`'lar hep aynı nav çubuğunun altında görünür. En sonda tanımlanan `<Route path="*" element={<BulunamadiSayfasi />} />` hiçbir yolla eşleşmeyen tanımsız her adres için render edilir.

» Komutlar

`npm install`	`react-router` bağımlılığını kurar
`npm run dev`	Vite geliştirme sunucusunu 5173 portunda başlatır
`uvicorn app.main:app --reload`	FastAPI sunucusunu 8000 portunda başlatır
`npx tsc -b`	TypeScript tiplerini derlemeden denetler
`python -m pytest -q`	Hf.6 testleri + yeni CORS testlerini çalıştırır

5 • İzleme **Nerede Kullanılır:** Promise reddedildi mi yoksa 200–500 arası bir durum kodu mu geldi → `response.ok` kontrol et, `try/catch`'e güvenme · Veri bekleniyor → `Yukleniyor` · İstek başarısız → `HataMesaji` · Liste boş → kısa "henüz yok" mesajı · Bağımlılık hızlı değişiyor → `AbortController` ile önceki isteği iptal et · JSON gövdeli `POST`/`PATCH` → tarayıcı önce preflight `OPTIONS` gönderir · Konsolda CORS hatası → sunucuda `allow_origins` kontrol et · kullanıcı tıklamasıyla gezinme → `Link` · kod içinden programatik gezinme → `useNavigate` · ortak nav/düzen → `Outlet` · tanımsız yol → `path="*"` · Adrese doğrudan gidiş/F5 → gerçek HTTP isteği (üretimde çözüm Hf.14)